

Name: BHAVYA GARG

Roll no: 23/38022

Sem: 5th

(DSE: Data Privacy)

1. Write a program to perform encryption and decryption using Caesar cipher (substitutional cipher).

```
import string

def encrypt(text, shift):
    result=""
    for i in range(len(text)):
        char=text[i]
        if(char.isupper()):
            result+=chr((ord(char)-ord('A')+shift)%26+ord('A'))
        elif char.islower():
            result+=chr((ord(char)-ord('a')+shift)%26+ord('a'))
        else:
            result+=char

    return result

def decrypt(text, shift):
    return encrypt(text, -shift)

def main():
    text="bbb"
    shift=2
    cipher_text=encrypt(text, shift)
    print(f"Encrypted text:{cipher_text}")
    plain_text=decrypt(cipher_text, shift)
    print(f"After decryption: {plain_text}")
main()
```

Output:

```
• Encrypted text:ddd  
  After decryption: bbb
```

2. Write a program to perform encryption and decryption using Rail Fence Cipher

(transpositional cipher)

```
def encrypt_rail_fence(text, key):  
    if key<2:  
        return text  
  
    rail=[['\n' for i in range(len(text))] for j in range(key)]  
  
    dir=False  
    row, col=0, 0  
    for i in text:  
        rail[row][col]=i  
        col+=1  
  
        if row==0 or row==key-1:  
            dir=not dir  
  
        if dir:  
            row+=1  
        else:  
            row-=1  
  
    result=[]  
    for i in range(key):  
        for j in range(len(text)):  
            if(rail[i][j]!='\n'):  
                result.append(rail[i][j])  
    return "".join(result)  
  
def decrypt(cipher, key):  
    if key<2:  
        return cipher  
    rail = [['\n' for i in range(len(cipher))] for j in range(key)]  
  
    dir_down = None
```

```

row, col = 0, 0

for i in range(len(cipher)):
    if row == 0:
        dir_down = True
    if row == key - 1:
        dir_down = False

    rail[row][col] = '*'
    col += 1

    if dir_down:
        row += 1
    else:
        row -= 1

index = 0
for i in range(key):
    for j in range(len(cipher)):
        if rail[i][j] == '*' and index < len(cipher):
            rail[i][j] = cipher[index]
            index += 1

result = []
row, col = 0, 0
for i in range(len(cipher)):
    if row == 0:
        dir_down = True
    if row == key - 1:
        dir_down = False

    if rail[row][col] != '\n':
        result.append(rail[row][col])
        col += 1

    if dir_down:
        row += 1
    else:
        row -= 1

return "".join(result)

def main():
    text="helloworld"

```

```

key=3
cipher_text=encrypt_rail_fence(text, key)
print(f"After encryption: {cipher_text}")
plain_text=decrypt(cipher_text, key)
print(f"After decryption: {plain_text}")

if __name__=="__main__":
    main()

```

Output:

```

After encryption: holelwrldo
After decryption: helloworld

```

3. Write a Python program that defines a function and takes a password string as input and

returns its SHA-256 hashed representation as a hexadecimal string.

```

import hashlib

def sha256_hash(password):
    encoded_password=password.encode()
    hash_object=hashlib.sha256(encoded_password)
    return hash_object.hexdigest()

def main():
    password=input("Enter a password: ")
    hashed_password=sha256_hash(password)
    print("SHA-256 hashed password: ", hashed_password)

main()

```

Output:

```

Enter a password: Hello
SHA-256 hashed password: 185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969

```

4. Write a Python program that reads a file containing a list of usernames and passwords, one pair per line (separated by a comma). It checks each password to see if it has been leaked in a data breach. You can use the "Have I Been Pwned" API (<https://haveibeenpwned.com/API/v3>) to check if a password has been leaked.

```
import hashlib
import requests

def check_password_pwned(password):
    sha1_hash = hashlib.sha1(password.encode()).hexdigest().upper()

    prefix = sha1_hash[:5]
    suffix = sha1_hash[5:]

    url = f"https://api.pwnedpasswords.com/range/{prefix}"
    response = requests.get(url)

    if response.status_code != 200:
        raise RuntimeError("Error fetching data from HaveIBeenPwned API")

    hash_list = response.text.split("\n")

    for line in hash_list:
        h, count = line.split(":")
        if h == suffix:
            return int(count)

    return 0

filename = "Directory.txt"

print("Checking passwords against HaveIBeenPwned...")

try:
    with open(filename, "r") as file:
        for line in file:
            line = line.strip()

            if not line:
                continue

            try:
                username, password = line.split(",")
```

```
except ValueError:
    print(f"Invalid line format: {line}")
    continue

count = check_password_pwned(password)

if count > 0:
    print(f"[LEAKED] User '{username}' → Password found {count}
times!")
else:
    print(f"[SAFE] User '{username}' → Password not found in
breaches.")

except FileNotFoundError:
    print(f"Error: File '{filename}' not found.")
```

Output:

```
Checking passwords against HaveIBeenPwned...
[LEAKED] User 'bob' → Password found 1220473 times!
[SAFE] User 'charlie' → Password not found in breaches.
[LEAKED] User 'bob' → Password found 1220473 times!
[SAFE] User 'charlie' → Password not found in breaches.
[LEAKED] User 'david' → Password found 4709969 times!
[LEAKED] User 'emma' → Password found 21969901 times!
[LEAKED] User 'fahad' → Password found 31369 times!
[LEAKED] User 'geeta' → Password found 1127198 times!
[LEAKED] User 'harsh' → Password found 5402944 times!
[LEAKED] User 'irfan' → Password found 1138064 times!
[LEAKED] User 'bob' → Password found 1220473 times!
[SAFE] User 'charlie' → Password not found in breaches.
[LEAKED] User 'david' → Password found 4709969 times!
[LEAKED] User 'emma' → Password found 21969901 times!
[LEAKED] User 'fahad' → Password found 31369 times!
[LEAKED] User 'bob' → Password found 1220473 times!
[SAFE] User 'charlie' → Password not found in breaches.
[LEAKED] User 'david' → Password found 4709969 times!
[LEAKED] User 'emma' → Password found 21969901 times!
[LEAKED] User 'bob' → Password found 1220473 times!
[SAFE] User 'charlie' → Password not found in breaches.
[LEAKED] User 'david' → Password found 4709969 times!
[LEAKED] User 'emma' → Password found 21969901 times!
[LEAKED] User 'fahad' → Password found 31369 times!
[LEAKED] User 'geeta' → Password found 1127198 times!
[LEAKED] User 'harsh' → Password found 5402944 times!
[LEAKED] User 'irfan' → Password found 1138064 times!
[LEAKED] User 'john' → Password found 460 times!
```

5. Write a Python program that generates a password using a random combination of words from a dictionary file.

```
import random

def generate_password(filename, num_words=4):
    try:
        with open(filename, "r") as file:
            words = [word.strip() for word in file.readlines() if word.strip()]

            selected_words = random.sample(words, num_words)
```

```
password = "-".join(selected_words)

return password

except FileNotFoundError:
    print("Error: Dictionary file not found.")
except ValueError:
    print("Error: Not enough words in dictionary to generate password.")

dictionary_file = "passwords"
password = generate_password(dictionary_file, num_words=4)
if password:
    print("Generated Password:", password)
```

Output:

```
PS C:\Users\Zeya\Documents\Request count> python
Generated Password: winter-book-forest-apple
PS C:\Users\Zeya\Documents\Request count>
```

6. Write a Python program that simulates a brute-force attack on a password by trying out

all possible character combinations.

```
import itertools
```

```
import time

def brute_force_attack(target_password):
    characters = "abcdefghijklmnopqrstuvwxyz"
    attempt_count = 0
    start_time = time.time()

    for length in range(1, len(target_password) + 1):
        for combo in itertools.product(characters, repeat=length):
            attempt = "".join(combo)
            attempt_count += 1

            if attempt == target_password:
                end_time = time.time()
                print(f"Password found: {attempt}")
                print(f"Total attempts: {attempt_count}")
                print(f"Time taken: {end_time - start_time:.4f} seconds")
                return

    print("Password not found.")

password=input("Enter the password to brute force : ")
brute_force_attack(password)
```

Output:

```
Enter the password to brute force : hello
Password found: hello
Total attempts: 3752127
Time taken: 0.4774 seconds
```